

Experiment No. – 4							
Date of Performance:	31-07-2026						
Date of Submission:	1-08-2026						
Conceptual Understanding (02)	Execution (1.5)	Problem Solving & Debugging(02)	Professional Conduct & Lab Ethics(02)	Output/ Result Accuracy (1.5)	Documentation & Result Analysis (01)	Experiment Total (10)	Sign with Date

Experiment No. 4 Study of Method Overloading

4.1 Aim: Develop a Java application for a Smart Calculator that performs arithmetic operations using method overloading.

4.2 Course Outcome: Develop Java programs using object-oriented programming principles such as classes, objects, constructors, and method overloading.

4.3 Learning Objectives: Use class and its methods in other subclasses, and to show how method overloading is an efficient way to write a program.

4.4 Requirement: Java development kit (JDK as per OS)

4.5 Related Theory:

Before understanding the method overloading first let's understand the polymorphism in Java. Polymorphism is the ability of an object to take on many forms. In other words, polymorphism in Java enables us to execute the same task in different ways. Method overloading in Java refers to the ability to define multiple methods within the same class that have the same name

but different parameter lists. This allows you to create methods that perform similar tasks but operate on different types of data or accept different numbers of parameters.

Let see an example: Suppose you want to find the summation of numbers provided by the user. But users can provide 2 parameters, 3 parameters or any number of parameters. Also users can provide different types of parameters.

```
Sum (int a, int b);  
Sum (int a, int b, int c);  
Sum (int a, float b);  
Sum (int a, int b, double c);
```

Here are the key points to understand about method overloading in Java:

Same Method Name: In method overloading, multiple methods within the same class have the same name.

Different Parameter Lists: Overloaded methods must have different parameter lists, which can differ in the following ways.

Number of parameters

Data types of parameters

Return Type: **Method overloading can't be based solely on the return type of the method.** Two methods with the same name and parameter list but different return types will result in a compilation error.

Compile-time Polymorphism: **Method overloading is an example of compile-time polymorphism** (also known as static polymorphism or early binding) because the decision about which method to call is made by the compiler based on the method signature.

Method Overloading: changing no. of arguments:

Two or more methods, first sum() method performs addition of two numbers and second sum() method performs addition of three numbers.

```
Class exp2  
{  
    static int sum(int a,int b)  
    {  
        Return a+b;  
    }  
    static int sum(int a,int b,int c)  
    {  
        Return a+b+c;  
    }  
}  
class Main  
{  
    public static void main(String[] args)  
    {  
        System.out.println(Adder.add(10,15));  
        System.out.println(Adder.add(05,10,20));  
    }  
}
```

Output:

25

35

Method Overloading: changing data type of arguments:

Two or more methods, first sum() method performs addition of two integer numbers and second sum() method performs addition of two double numbers.

Class exp2

```
{
static int sum(int a, int b)
    {
        Return a+b;
    }
static double sum(double a, double b)
    {
        return a+b;
    }
}
class Main
{
public static void main(String[] args)
    {
        System.out.println(Adder.add(10,10));
        System.out.println(Adder.add(10.5,20.7));
    }
}
```

Output:

20

31.2

Overloading the java main() method:

We can have number of main methods in a class. Java virtual machine calls only main() method which receives string array as arguments.

Class exp2

```
{
public static void main(String[] args)
    {
        System.out.println("main with String[]");}
public static void main(String args)
    {
        System.out.println("main with String");
    }
public static void main()
    {
        System.out.println("main without args");
    }
}
```

Method Overriding in Java:

Method Overriding can only be implemented in Super class sub class hierarchy. Method Overriding conveys the same name, same parameters, with different implementation but in super class sub class hierarchy. Method overriding is runtime polymorphism.

Different super class sub class hierarchy in Java:

Single Inheritance:

Class A

```
{
    //Class A data
    //Class A methods
}
```

Class B extends A

```
{
    //Class A data
    //Class B methods
}
```

Multilevel Inheritance:

Class A

```
{
    //Class A data
    //Class A methods
}
```

Class B extends A

```
{
    //Class B data
    //Class B methods
}
```

Class C extends B

```
{
    //Class C data
    //Class C methods
}
```

Hierarchical Inheritance:

Class A

```
{
    //Class A data
    //Class A methods
}
```

Class B extends A

```
{
    //Class B data
    //Class B methods
}
```

Class C extends A

```
{
    //Class C data
}
```

```

        //Class C methods
    }

Method Overriding Example:
class Vehicle
{
    void run()
    {
        System.out.println("Vehicle is running");
    }
}
class Bike extends Vehicle
{
    void run()
    {
        System.out.println("Bike is running safely");
    }
}
public static void main(String args[])
{
    Bike2 obj = new Bike2();
    obj.run();
}
}

```

4.6 Procedure:

Create a java class with two or more methods of the same name but different in number of parameters or different in type of parameters or different return type. Write different method logic depending on the type of method (Differentiated based on parameters of type). Call these methods with the help of an object to show method overloading.

4.7 Program and Output:

Code:

```

import java.util.Scanner;

class SmartCalculator {

    void calculate(int a, int b) {
        System.out.println(a + b);
    }

    void calculate(int a, int b, int c) {
        System.out.println(a + b + c);
    }

    void calculate(double a, double b) {

```

```

        System.out.println(a + b);
    }

    void calculate(int a, int b, boolean multiply) {
        System.out.println(a * b);
    }

    void calculate(double a, double b, boolean multiply) {
        System.out.println(a * b);
    }
}

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        SmartCalculator obj = new SmartCalculator();

        int a = sc.nextInt();
        int b = sc.nextInt();

        int c = sc.nextInt();
        int d = sc.nextInt();
        int e = sc.nextInt();

        double f = sc.nextDouble();
        double g = sc.nextDouble();

        int h = sc.nextInt();
        int i = sc.nextInt();

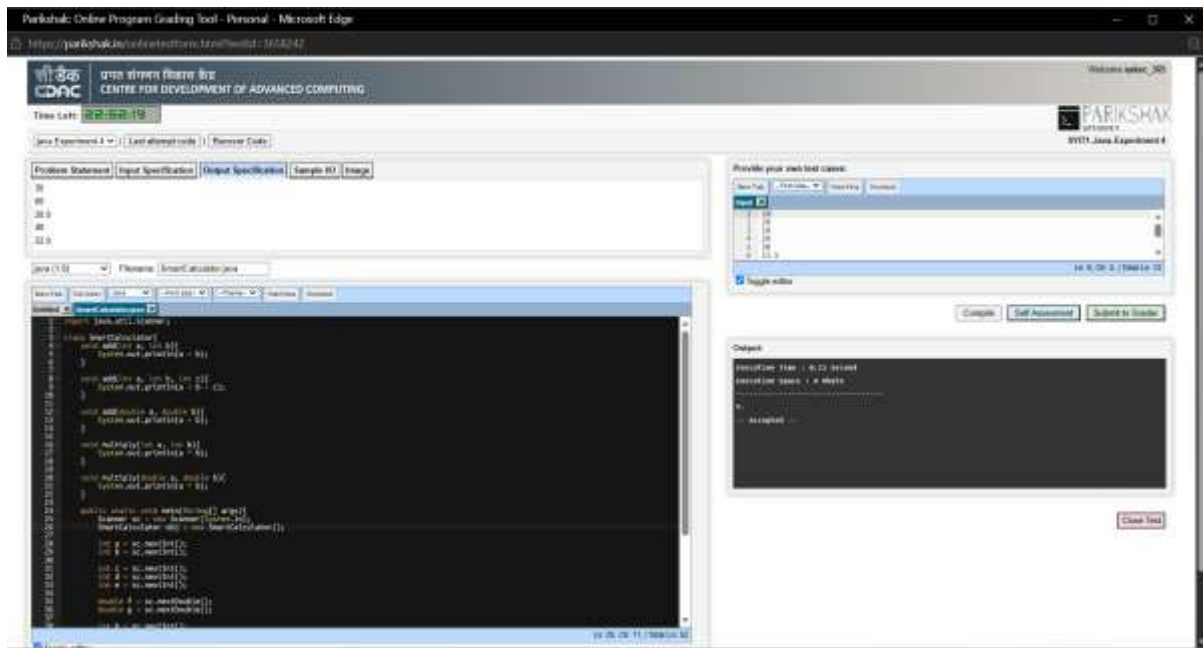
        double j = sc.nextDouble();
        double k = sc.nextDouble();

        obj.calculate(a, b);
        obj.calculate(c, d, e);
        obj.calculate(f, g);
        obj.calculate(h, i, true);
        obj.calculate(j, k, true);

        sc.close();
    }
}

```

Output:



4.8 Conclusion:

Method overloading was successfully implemented in the Smart Calculator application. Multiple methods with the same name but different parameter lists were used to perform different arithmetic operations. This demonstrated compile-time polymorphism, improved code readability, and allowed similar operations to be performed efficiently using a single method name.

4.9 Questions:

1. What is Method overloading in java.

Ans:

Method overloading is a feature in Java that allows multiple methods in the same class to have the same name but different parameter lists (different number or type of parameters). It is an example of compile-time polymorphism.

2. What is application of method overloading

Ans:

Method overloading is used to perform similar operations with different types or numbers of inputs. It improves code readability, reduces the need for multiple method names, and increases code reusability.

3. What is polymorphism and it can achieved in java other than method overloading

Ans:

Polymorphism is the ability of an object to perform the same operation in different ways. Besides method overloading (compile-time polymorphism), it can also be achieved through **method overriding**, which provides runtime polymorphism using inheritance.